top, htop, iostat -x 5, sudo apt-get install sysstat, netstat -x, netstat -i, sar -n DEV 3, strace df -h, sudo strace -p PID (PID de ssh), strace -i df -h,

Optimization and Profiling

Msc. Ramiro Andrés Barrios Valencia

Performance Metrics

- FLOPS, or FLOP/S: FLoating-point Operations Per Second
- MFLOPS: MegaFLOPS, 10^6 flops
- GFLOPS: GigaFLOPS, 10^9 flops, home PC
- TFLOPS: TeraGLOPS, 10^12 flops, present-day supercomputers (www.top500.org)
- PFLOPS: PetaFLOPS, 10^15 flops, by 2011
- EFLOPS: ExaFLOPS, 10^18 flops, by 2020
- MIPS=Mega Instructions per Second = MegaHertz (if only 1IPS)

Note: von Neumann computer -- 0.00083 MIPS

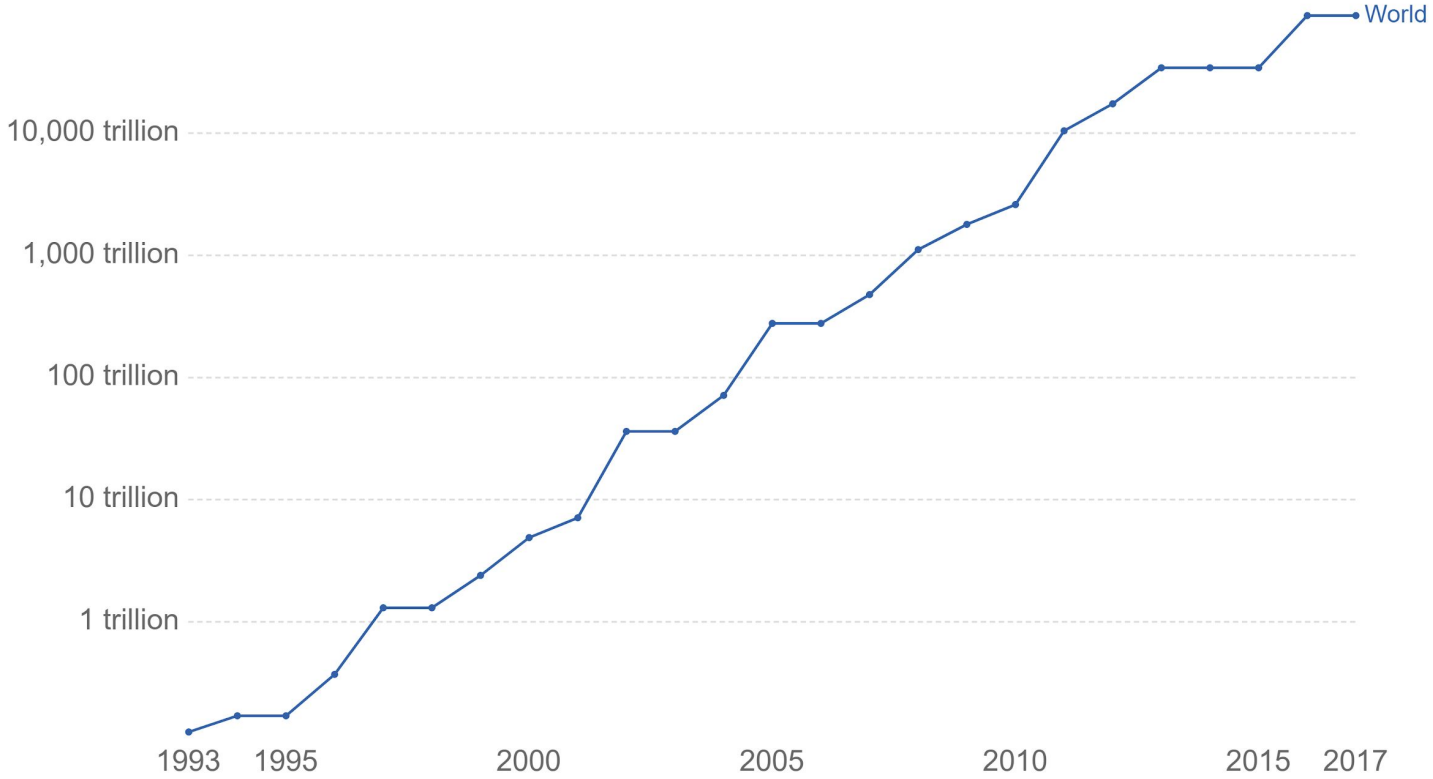
Performance Metrics

- Theoretical peak performance R_theor: maximum FLOPS a machine can reach in theory..
- Clock_rate*no_cpus*no_FPU/CPU:
  - 3GHz, 2 cpus, 1 FPU/CPU à R_theor=3x10^9 * 2 = 6 GFLOPS
- Real performance R_real: FLOPS for specific operations, e.g. vector multiplication.
- Sustained performance R_sustained: performance on an application, e.g. CFD

R_sustained << R_real << R_theor -- Not uncommon R_sustained < 10%R_theor

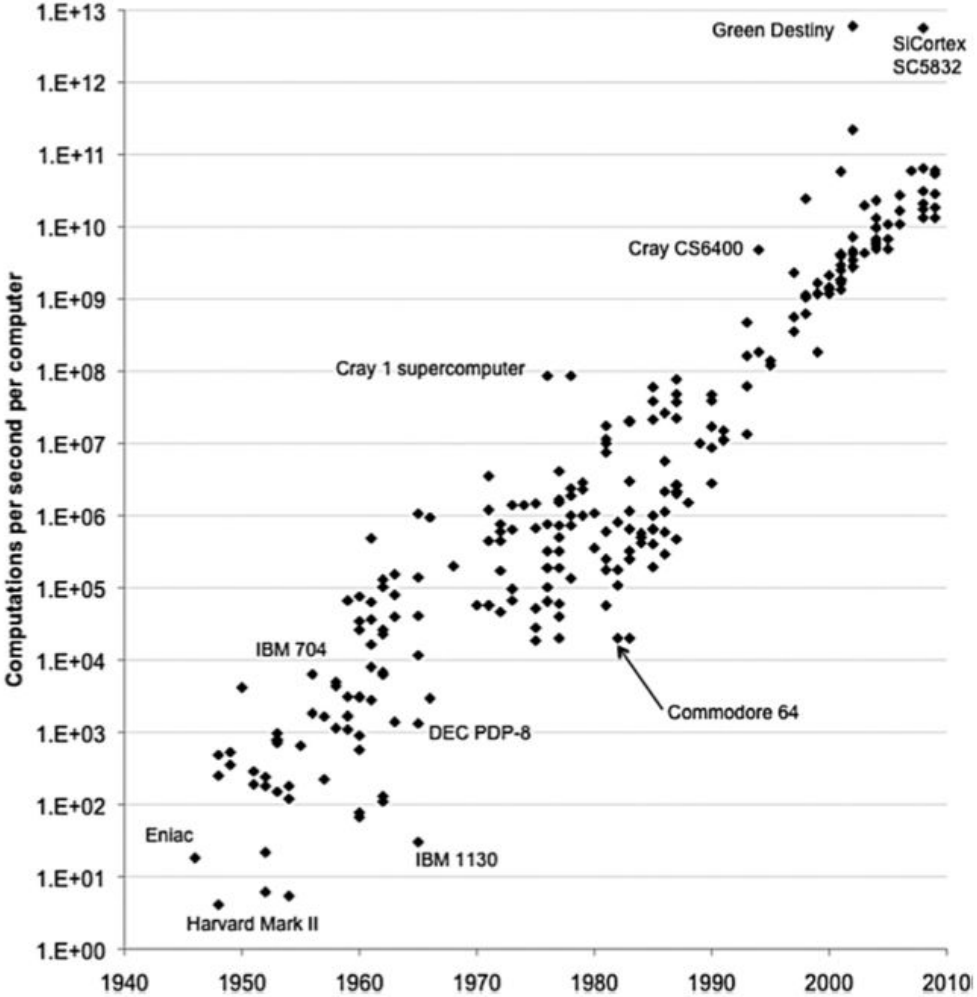
Supercomputer Power (FLOPS)

The growth of supercomputer power, measured as the number of floating-point operations carried out per second (FLOPS) by the largest supercomputer in any given year. (FLOPS) is a measure of calculations per second for floating-point operations. Floating-point operations are needed for very large or very small real numbers, or computations that require a large dynamic range. It is therefore a more accurate measured than simply instructions per second.



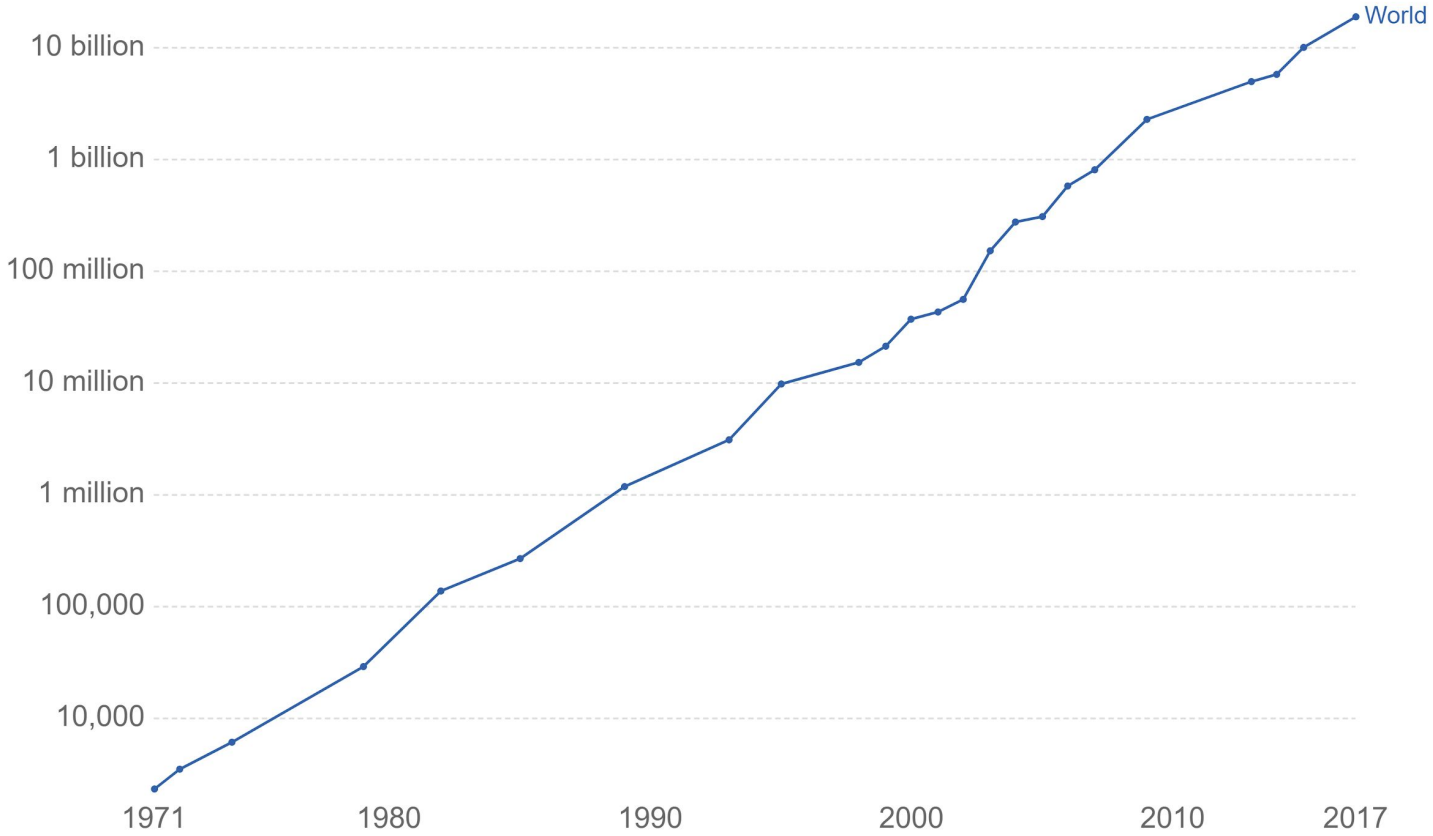
Source: TOP500 Supercomputer Database

CC BY


Computations per second per computer
1.E+13
1.E+12
1.E+11
1.E+10
1.E+09
1.E+08
1.E+07
1.E+06
1.E+05
1.E+04
1.E+03
1.E+02
1.E+01
1.E+00
1940
1950
1960
1970
1980
1990
2000
2010
Green Destiny
SiCortex
SC5832
Cray CS6400
Cray 1 supercomputer
IBM 704
Commodore 64
DEC PDP-8
Eniac
IBM 1130
Harvard Mark II

Moore's Law: Transistors per microprocessor

Number of transistors which fit into a microprocessor. This relationship was famously related to Moore's Law, which was the observation that the number of transistors in a dense integrated circuit doubles approximately every two years.



Source: Karl Rupp. 40 Years of Microprocessor Trend Data.

CC BY

Why to profile?

- Evaluate performance

- Find the performance bottlenecks
  - Inefficient programming
  - Memory I/O bottlenecks
  - Parallel scaling

Tools categories

- Hardware counters

  Count events from CPU perspective (# of flops, memory loads, etc)

  Usually need Linux kernel module installed

- Statistical profilers (sampling)

  Interrupt program at given intervals to find what routine/line the program is in

- Event based profilers (tracing)

  Collect information on each function call

Simple Profiling

- Time program runtime

  Get an idea on time to run and parallel scaling

- Serial profiling
  - Discover inefficient programming
  - Computer architecture slowdowns
  - Compiler optimizations evaluation
  - gprof:

    Trick how to get gprof to work in parallel:
    https://shwina.github.io/profiling-parallel/

```
[sas@discovery]$ icc -g -pg -openmp-stubs -o matmul_par matmul_par.c
matmul_par.c(60): warning #161: unrecognized #pragma
 #pragma omp parallel for private(tmp, i, j, k)
 ^
[sas@discovery]$ ./matmul_par
 Order 1000 multiplication in 13.498900 seconds
 1 threads
 Hey, it worked
 all done

[sas@discovery] ls -ls gmon.out
4 -rw-rw-r-- 1 sas sas 1317 Feb 17 11:27 gmon.out

[sas@discovery] gprof -l matmul_par >gprof.out

[sas@discovery] more gprof.out

Flat profile:

Each sample counts as 0.01 seconds.
 % cumulative self self total
 time seconds seconds calls Ts/call Ts/call name
 94.39 18.06 18.06 main (matmul_par.c:68 @ 400aea)
 5.72 19.15 1.09 main (matmul_par.c:66 @ 400b22)
 0.05 19.16 0.01 main (matmul_par.c:50 @ 4009f8)
 0.05 19.17 0.01 main (matmul_par.c:54 @ 400a57)
 0.05 19.18 0.01 main (matmul_par.c:70 @ 400b30)
 0.05 19.19 0.01 main (matmul_par.c:85 @ 400c39)
.
```

Open Source Tools

- Vendor based
  - AMD CodeAnalyst
  - CrayPat is a performance analysis tool offered by Cray for the XC platform.
  - Intel Inspector
- Community based
  - perf: Hardware counter collection, part of Linux
  - oprofile: profiler
  - drawback: harder to analyze the profiling results

HPC OS Tools

- HPC Toolkit

  A few years old, did not find it as straightforward to use

- TAU

  Lots of features, which makes the learning curve slow

- Scalasca

  Developed by European consortium, did not try yet

Interpreted Language Profiling

With increased use of interpreted languages, their performance is becoming important

- Matlab

  Profiling ecosystem in the IDE

- Python

  Python modules or IDEs

- R

  Profiling libraries or RStudio

Matlab

- profile command turns on/off profiling
- Profile is then displayed in the IDE

  Click on each function to show line-by-line profile

- Performance improvement strategies

  https://www.mathworks.com/help/matlab/matlab_prog/techniques-for-improving-performance.html

Python

- profile and cProfile modules
  - Text based output, optional format with pstats, analysis with Stats
- Plethora of other tools
  - E.g. line profiling with line_profiler
- Some IDEs display profiles
  - Spyder

R

- Rprof function to profile
- summaryRprof to display
- RStudio has a profile interface called profviz
- Performance improvement strategies http://adv-r.had.co.nz/Profiling.html